Multimodal AI Product Evaluation

Getting Started

This guide tells you where to start, how to navigate, and what to do next — depending on where you are in your work.

Note: Please refer to the Templates and Visual Artifacts in this reference document: https://docs.google.com/document/d/1ycQbWhKPiR1BsZbiXkmnkG-5VoruHxfsEHCJFG8EtOk/edit?usp=sharing

---

Who Are You? Start Here.

Before reading anything else, find your situation in this table. It will tell you exactly where to go.

You are...	Your entry point	Estimated Time to first value
A PM who needs to make a ship/hold decision now	T-14 (Decision Memo) → T-10 (Gate Thresholds) → Rubric in Ch 14	1 day
An ML engineer asked to "build evals"	Ch 4 (Error Analysis) → T-05–T-09 (Evaluator Prompts) → T-11 (Regression Suite)	2 weeks
A founder evaluating a product before launch	T-01 (Evaluation Brief) → Ch 3 (Surface Map) → Ch 20 (12-Week Plan)	3 days
An AI Reliability Lead inheriting a broken program	Appendix D (Red Flags) → T-18 (Debt Register) → Ch 20	1 week
A new team member onboarding to an existing program	VA-01 (Roadmap) → Ch 1 → Ch 3 → Ch 4	2 days
Just starting with no evaluation program yet	Ch 1 → Ch 2 → Ch 3 → Ch 4 → Ch 20	12 weeks

Something broke in production right now	T-13 (Driver Analysis) → Ch 13 (Fix Surfaces) → T-17 (Backlog)	Today
Running a quarterly program health check	T-18 (Debt Register) → Ch 16 → Appendix D	1 week

---

The Mindset Before Everything Else

These four commitments are the prerequisite mental model for every activity in MultiModal AI Evaluations. Practitioners who skip them tend to build evaluators for the wrong definition of quality, fix symptoms instead of causes, and anchor their standards to what they already built.

Commitment 1 — Per-Modality, Per-Contract Thinking

Never ask "is this output good?" Ask five separate questions — one per contract. The moment you collapse them into a single score, you lose the visibility that makes improvement possible. A 94% overall quality score can mask a complete ingestion failure on scanned documents that affects 15% of your users. The five contracts are always separate:

Contract	The Core Question
Ingestion Fidelity	Did the pipeline faithfully extract what was in the source?
Faithfulness	Is the output traceable to the sources?
Factuality	Is the output correct about the real world?
Cross-Modal Consistency	Do the output modalities agree with each other?
Output Quality	Is the production quality adequate per medium?

Commitment 2 — Pipeline-First Diagnosis

When something goes wrong, resist the impulse to fix the prompt. Approximately one-third of multimodal failures originate at the ingestion layer — before the AI model is involved at all. No prompt instruction can compensate for a PDF parser that drops figure captions. The model never received the content. Trace every failure to its origin stage before proposing any intervention.

Commitment 3 — Reading Real Traces

Automated metrics multiply human judgment — they do not replace it. The practitioner who has closely read 150 real production traces will consistently outperform one who has designed

sophisticated automated evaluators but never looked carefully at what users actually upload and what the system actually does with it. Plan for 60–80% of your development time to go to error analysis. This will feel wrong while you are doing it. It is not.

Commitment 4 — Pre-Committed Thresholds

Teams that set quality bars after seeing their scores are not setting standards — they are describing what they already built. Every threshold in this framework must be agreed on and signed off *before* any evaluation runs. This is the hardest discipline to maintain and the one most teams abandon first. If your current performance is below the bar you set, that is a product decision — not a reason to lower the bar.

What This Guide Is

This is a guide to an **operational evaluation framework** for Multimodal AI products that process or produce multimodal content — PDFs, audio, images, slides, video, or any combination.

It is:

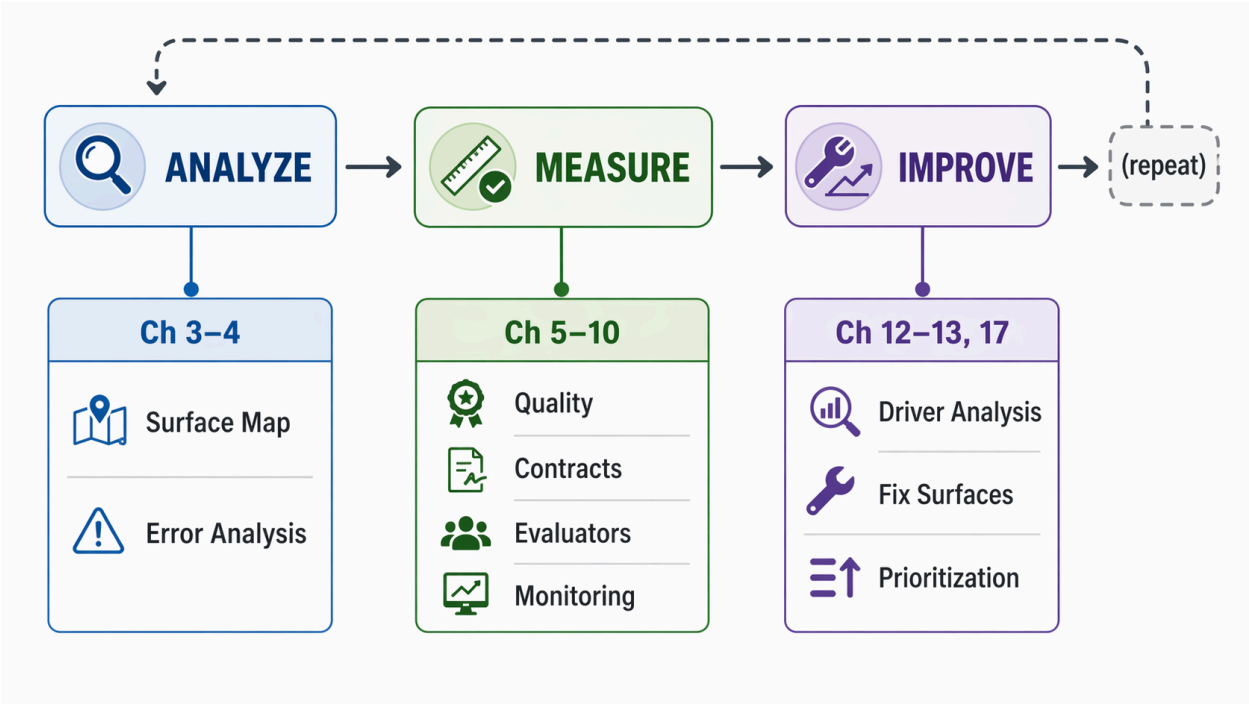
- A repeatable discipline for teams building multimodal AI products in production
- A structured way to find failures before users do, and fix them at the right place in the pipeline
- A governance system that prevents evaluation programs from silently decaying

The AMI Evaluation Loop

The AMI Cycle Is the Spine

Every activity in this framework belongs to one of three phases:

```
ANALYZE ──► MEASURE ──► IMPROVE ──► (repeat)
   |           |           |
Ch 3–4       Ch 5–10     Ch 12–13, 17
Surface Map    Quality       Driver Analysis
Error Analysis Contracts     Fix Surfaces
               Evaluators    Prioritization
               Monitoring
```



You do not run these phases once. You cycle through them continuously. Each cycle tightens your taxonomy, improves your evaluators, and reduces the time between a failure appearing in production and a fix landing.

Step-by-Step Process

Pre-launch teams: You do not need production data to start. See the pre-launch callout in Phase 2 for your synthetic trace strategy.

Phase 1 — Foundation

Templates used this phase: T-01 (Evaluation Brief) · T-03 (Architecture Trade-off Worksheet) · T-04 (Logging Schema)

What you're doing: Building the shared understanding that makes every subsequent activity accurate. Nothing built in Phases 2–3 is reliable without this foundation.

Step 1: Name Your Judge Panel

You need four people — a Content Judge, an Ingestion Judge, an Audio Judge (if your product produces audio), and a Visual Judge (if your product produces visuals).

Minimum viable Judge Panel: two people. Name a tiebreaker.

Do this before anything else. The panel is the anchor for every rubric, every threshold, and every quality bar in the program. Without named judges, your evaluation program has no definition of "good."

Role	Jurisdiction	Typical background
Content Judge	Factual accuracy, claim grounding, source representation	PM, domain expert, senior user
Ingestion Judge	Parsing, transcription, figure extraction	ML engineer, data engineer
Audio Judge	Voice naturalness, pacing, pronunciation, prosody	Audio producer, UX researcher
Visual Judge	Layout, node clarity, taxonomic structure	Designer, visual communicator
Tiebreaker	Final decision authority when panel disagrees	Founding team, senior engineer

Step 2: Fill In and Sign the Evaluation Brief (T-01)

The evaluation brief forces your team to answer hard questions about what "good" means *before* those answers get buried in code and *before* different team members discover they define success differently. It takes one hour to fill in and saves three months of rework.

⚠️ **Critical:** Set every threshold *before* running any evaluations. Teams that measure first and set thresholds after anchor to what's currently achievable, not what users need.

Step 3: Install One Observability Tool

Pick one platform and deploy it completely. Strong options: Braintrust (best for dataset versioning and offline evaluation), LangSmith (best if you're using LangChain), Arize Phoenix (best for embedding drift), Langfuse (best for HIPAA/SOC2 requirements).

Verify it captures ingestion metadata — not just inputs and outputs. Two partially-instrumented platforms produce worse signal than one fully-instrumented one.

Step 4: Run the Ten-Trace Exercise

Block four hours with your Judge Panel and one engineer. Pull 10 real (or realistic synthetic) production traces. For each trace, walk through these six questions in order:

1. What did the user actually upload? *(Open the original files — not the extracted text.)*

2. What did the ingestion layer produce? *(Compare extracted representation to the original.)*
3. What was indexed, and in what modality distribution?
4. What was retrieved? *(Which chunks? From which modality types?)*
5. What context was assembled for the model? *(Token counts per section?)*
6. Where do you first notice something wrong when consuming the output as a user?

For each problem you find, trace it back to the earliest stage where it first appeared — not where it became visible. A mispronounced statistic in the audio became visible at Stage 7. It likely originated at Stage 1 when the parser dropped the figure caption.

This exercise produces your first-draft surface map and reveals your logging blind spots simultaneously.

✓ **Phase 1 Is Complete When:**

- ☐ Four judges named with modality jurisdictions documented
- ☐ Evaluation Brief (T-01) filled in and signed by all judges, PM, and Eng Lead
- ☐ All thresholds set *before* any evaluations run
- ☐ At least two zero-tolerance events named per contract
- ☐ One observability tool installed and capturing ingestion metadata
- ☐ Ten-Trace Exercise completed; first-draft surface map on the team wiki
- ☐ Logging schema (T-04) reviewed; blind spots identified and sprint-planned

Phase 2 — Error Analysis

Templates used this phase: T-17 (Backlog Prioritization — first draft) · T-11 (Regression Suite Coverage Audit — initial gap analysis)

What you're doing: Building the shared vocabulary of failure that every evaluator, every routing decision, and every fix will reference. This is the most important phase in the program. Do not abbreviate it.

Budget two 2-hour sessions per week, 15–20 traces per session, targeting **150 traces total**. In every project that ships a reliable AI product, 60–80% of development time goes here.

Pre-Launch? Here Is Your Synthetic Trace Strategy

You do not need production traces to start error analysis. Create synthetic inputs that deliberately stress the system:

- Scanned PDFs with handwritten annotations
- Multi-column academic papers with embedded figures and captions
- Audio recordings with domain-specific acronyms and proper nouns
- Documents mixing primary and non-primary languages
- Files at the edge of supported size limits
- Slide decks with data-heavy charts and minimal explanatory text

Your dogfood distribution is three standard deviations cleaner than production will be. Deliberately correct for this when creating synthetic inputs.

Label all synthetic traces clearly in your taxonomy. Replace them with real production traces as they become available. Your taxonomy built on synthetic data will need to be validated against real traffic — plan a re-labeling pass after your first 30 days in production.

Step 1: Traces Coding (30–40 traces)

For each trace, write free-text notes about everything that went wrong. Answer two questions per note — and *only* these two:

- Which modality did this failure *appear in*?
- Which stage did it *originate from*?

Do not categorize yet. Just describe with precision.

Good notes look like:

> *"Parser dropped the caption under Figure 3. The audio later stated the general topic but gave no specific values — because it never received them."* → Origin: Stage 1 | Appears in: Audio

Notes to avoid: *"Audio quality issue."* — *"Hallucination."* — *"Map looks wrong."* These are symptoms, not causes. You cannot build an evaluator for "audio quality issue."

Step 2: Tagged Coding (50–80 traces)

After 50+ traces, cluster your open-coded notes into named failure tags. Target 10–15 primary codes. Organize by pipeline stage. Each tag carries two facets: what went wrong, and where it originated.

Starter taxonomy (adapt to your product):

INGESTION LAYER
- ingestion.[source_type].dropped_caption
- ingestion.[source_type].layout_scramble
- ingestion.ocr.scanned_drop
- ingestion.asr.term_miss
- ingestion.asr.silence_hallucination

RETRIEVAL + GROUNDING
- retrieval.modality_bias
- retrieval.visual_chunk_dropped
- grounding.faithfulness_break
- grounding.speaker_attribution
- grounding.cross_source_merge

GENERATION
- generation.fabricated_quote
- generation.invented_statistic

CROSS-MODAL
- cross_modal.output_mismatch
- cross_modal.speaker_drift

OUTPUT SYNTHESIS
- output.audio.mispronunciation
- output.audio.prosody_flat
- output.visual.layout_overlap
- output.visual.taxonomic_error
- output.visual.concept_gap

SAFETY
- safety.pii_in_output
- safety.copyright_reproduction

Step 3: Re-label, Count, Route (remaining traces to 150)

Return to trace one and re-label every failure with your axial codes — *without* looking at your original open-coded notes. Then:

- Count failures by pipeline stage
- Count failures by tag (identify your top 10)
- Document causal chains for all multi-tag traces
- Route each tag to its first-line and second-line fix surface

> ⚠️ **The Causal Chain Rule:** For every trace with two or more failure tags, ask whether the failures are causally chained. An ingestion fix that eliminates a dropped-caption failure also resolves downstream faithfulness breaks and audio mispronunciations in the same chain. One fix; multiple failures resolved. Document each chain before proposing any intervention.

✓ **Phase 2 Is Complete When:**

- ☐ 150 traces reviewed (or 80 minimum with synthetic supplement, clearly labeled)
- ☐ Taxonomy published to team wiki with 10–15 named tags
- ☐ Every tag carries both an appearance modality AND an origin stage
- ☐ Causal chains documented for all multi-tag traces
- ☐ Routing table maps every tag to first-line and second-line fix surfaces
- ☐ Count tables produced: by stage and by tag (top 10)
- ☐ At least one high-score fix identified using a first-pass priority formula estimate and assigned to an owner

Phase 3 — Build the Measurement Layer

Templates used this phase: T-02 (Quality Statement) · T-05–T-09 (Evaluator Prompts) · T-11 (Regression Suite Coverage Audit) · T-12 (CI Gate Policy)

What you're doing: Turning human judgment into automated evaluators that your automated CI system can enforce. Nothing in this phase is reliable without the taxonomy and causal chains from Phase 2.

Step 1: Human Labels First

Take 100–150 outputs from your error analysis traces. Have the relevant SME apply their verdict to each one. Split:

- **60%** → build the evaluator
- **20%** → validate it (measure kappa)
- **20%** → holdout (never touch until you think you're done)

Using the holdout early is data leakage. It produces evaluators that look better than they are. One run, at the end, final.

Step 2: Build Your LLM-as-a-Judge Evaluators (T-05 through T-09)

Build one LLM-as-a-Judge evaluator per contract, starting with your most frequent failure tag. Every evaluator prompt needs four elements:

- A precise task description
- Strict **binary** pass/fail definitions — not 1–5 scales
- 3–5 calibration examples from *your own product's* actual failures
- Structured JSON output with a reasoning field

Use a **different model family** for judging than for generating. Same-family judges produce self-enhancement bias that inflates your scores.

Step 3: Validate Before Deploying

Measure Cohen's Kappa between your evaluator and your judge on the validation split. Cohen's Kappa answers the question: **"Does my automated evaluator agree with my human judge — or is it just getting lucky?"**

Kappa = (Actual agreement − Agreement by chance) ÷ (Perfect agreement − Agreement by chance)

- **Kappa = 1.0** → Your evaluator agrees with your judge perfectly, every time.
- **Kappa = 0.0** → Your evaluator is doing no better than random chance.
- **Kappa < 0.0** → Your evaluator is actually *worse* than random chance (a sign something is very wrong).

Kappa	What It Means	Action
Below 0.60	Substantial disagreement	Rewrite rubric from scratch. Do not deploy.
0.60–0.69	Moderate agreement	Revise calibration examples. Not yet deployable.
0.70–0.79	Acceptable	Deploy for monitoring only. Not for CI gating.
0.80+	Strong agreement	Required before blocking CI merges.

Slice analysis is mandatory. An evaluator with overall kappa 0.82 that has kappa 0.43 on audio-specific claims has a problem hidden by a text-heavy test set. Report kappa separately for each content type.

Sensitivity test: Plant 10 known-bad outputs into your validation set. The evaluator must catch all 10. Any miss means your rubric has a gap.

Step 4: Wire CI Gating (T-11, T-12)

Build your regression suite: 60–100 cases minimum, at least 5 per taxonomy tag, media blobs registered with SHA-256 hashes. Wire the CI gate. Document your false positive policy (T-12) before the gate fires for the first time — not after.

✓ **Phase 3 Is Complete When:**

- ☐ Human labels collected for all five contracts (100–150 per evaluator)
- ☐ 60/20/20 split maintained; holdout untouched
- ☐ At least one evaluator deployed with kappa ≥ 0.70 documented
- ☐ Slice analysis run on every deployed evaluator
- ☐ Sensitivity test passed for every deployed evaluator (10/10 known-bad caught)
- ☐ Regression suite at 60+ cases with ≥ 5 cases per taxonomy tag
- ☐ Media blobs registered with SHA-256 manifest
- ☐ CI gate live and blocking real merges
- ☐ False positive policy (T-12) documented and communicated to engineering team

Adapting for Your Team Size

The framework as written assumes a 3–8 person team. Here is how to adapt it for different contexts.

Solo Practitioner or Two-Person Team

You cannot maintain four separate Judge Panel roles. Consolidate: one person holds Content + Ingestion; the second holds Audio + Visual. Skip the tiebreaker — disagreements defer to your most experienced user.

For error analysis: target 80 traces instead of 150. For the regression suite: target 40 cases minimum, 3 cases per tag minimum. For the 12-week plan: budget 14–16 weeks. Phase 2 takes longer when one person is doing all the labeling.

Everything else in the framework applies without modification. The templates scale down gracefully — fill in what applies to your product and leave modality-specific rows blank.

Three to Six People (Standard Early-Stage Team)

The full 12-week plan as written fits this team size. Assign Judge Panel roles as functional responsibilities, not additional headcount:

- Ingestion Judge → typically your ML or data engineer
- Content Judge → typically your PM or domain expert
- Audio/Visual Judge → whoever has the most design or production media experience
- Tiebreaker → founding team member or senior engineer

Larger Team or Enterprise Context

The Judge Panel becomes a representative committee, not four individuals. Budget 16–20 weeks for the first cycle — organizational alignment at each milestone takes longer than the technical work. The Evaluation Debt Register (T-18) becomes a formal quarterly deliverable reviewed by engineering leadership.

You will also need:

- A Prompt Review Board with formal quorum rules and documented approval workflow
- A named AI Reliability Lead with explicit ownership over the full evaluation program
- A formal false positive policy (see T-12) reviewed before the CI gate is activated
- An escalation path for zero-tolerance events that goes above the team level

Getting Organizational Buy-In

The hardest part of this framework is not the evaluation work — it is convincing your organization that 60–80% of development time should go to error analysis *before* building evaluators. Three arguments that consistently work:

The cost argument. One production incident caused by an undetected ingestion failure typically costs 10–50× the total investment in the evaluation program that would have caught it. Quantify your last production incident — time to detect, time to fix, users affected, trust damage. The math usually closes the argument.

The speed argument. Teams without a systematic evaluation program spend enormous time debugging failures they cannot characterize, proposing fixes that address symptoms instead of root causes, and re-encountering resolved failures because they were never added to a regression suite. The 12-week upfront investment is typically recovered in the first quarter of sustained operation.

The trust argument. In regulated industries — finance, healthcare, legal — the question is not whether you evaluate, but whether you can demonstrate that your evaluation program is systematic, documented, and maintained. This framework produces that documentation as a byproduct of normal operation.

The fastest path to buy-in: Do not present the full framework. Present the Evaluation Brief (T-01) first. Ask your manager to co-sign it. The act of filling in the brief — and discovering that your team disagrees about quality thresholds — is itself the argument for why the program is necessary. Every team that has tried this has found at least one significant disagreement they did not know they had.